

1. schedule_manager.py

ဒီကောင်က Schedule data ကို စီမံမယ်။

```
class ScheduleManager:
```

```
    def __init__(self):
```

```
        self.schedules = []
```

```
        self.next_id = 1
```

```
    # Create
```

```
    def create_schedule(self, name, task_id, start_date, start_time, repeat,
after_action, enabled=True):
```

```
        schedule = {
```

```
            "id": self.next_id,
```

```
            "name": name,
```

```
            "task_id": task_id,
```

```
            "start_date": start_date,
```

```
            "start_time": start_time,
```

```
            "repeat": repeat,
```

```
            "after_action": after_action,
```

```
            "enabled": enabled}
```

```
        self.schedules.append(schedule)
```

```
        self.next_id += 1
```

```
        return schedule
```

```
    # Get
```

```
    def get_schedules(self):
```

```
        return self.schedules
```

```
    # Delete
```

```
    def delete_schedule(self, schedule_id):
```

```
        self.schedules = [
```

```

        schedule

    for schedule in self.schedules

        if schedule["id"] != schedule_id

    ]

# Enable / Disable

def set_enabled(    self,    schedule_id,    enabled    ):

    for schedule in self.schedules:

        if schedule["id"] == schedule_id:

            schedule["enabled"] = enabled

            return True

    return False

```

အခုတော့ Database မထည့်သေးဘူး။ Memory ထဲပဲ သိမ်းမယ်။

2. schedule_dialog.py

User က Schedule information ထည့်မယ့် Dialog။

```

from PyQt6.QtCore import pyqtSignal

from PyQt6.QtWidgets import ( QDialog, QVBoxLayout, QLineEdit,
    QDateEdit, QTimeEdit, QComboBox, QCheckBox, QPushButton, QLabel,)

class ScheduleDialog(QDialog):

    schedule_created = pyqtSignal(dict)

    def __init__(    self,    task_id=None,    parent=None    ):

        super().__init__(parent)

        self.task_id = task_id

        self.setWindowTitle(    "Add Schedule"    )

        self.resize(400, 400)

        self.setup_ui()

    def setup_ui(self):

        layout = QVBoxLayout(self)

```

```

# Name

layout.addWidget(      QLabel("Schedule Name")      )

self.name_input = QLineEdit()

self.name_input.setPlaceholderText(      "Night Download"      )

layout.addWidget(      self.name_input      )

# Start Date

layout.addWidget(      QLabel("Start Date")      )

self.date_input = QDateEdit()

self.date_input.setCalendarPopup(      True      )

self.date_input.setDate(      self.date_input.date()      )

layout.addWidget(      self.date_input      )

# Start Time

layout.addWidget(      QLabel("Start Time")      )

self.time_input = QTimeEdit()

layout.addWidget(      self.time_input      )

# Repeat

layout.addWidget(      QLabel("Repeat")      )

self.repeat_combo = QComboBox()

self.repeat_combo.addItem(      "Once",      "Daily",      "Weekly",
"Monthly" ])

layout.addWidget(      self.repeat_combo      )

# After Action

layout.addWidget(      QLabel("After Download")      )

self.after_action_combo = QComboBox()

self.after_action_combo.addItem(      "Nothing", "Shutdown", "Sleep", "Exit Application"
])

layout.addWidget(      self.after_action_combo      )

# Enabled

self.enabled_check = QCheckBox(      "Enabled"      )

```

```

self.enabled_check.setChecked(    True    )

layout.addWidget(    self.enabled_check    )

# Buttons

self.save_button = QPushButton(    "Save"    )

self.cancel_button = QPushButton(    "Cancel"    )

layout.addWidget(    self.save_button    )

layout.addWidget(    self.cancel_button    )

self.save_button.clicked.connect(    self.save    )

self.cancel_button.clicked.connect(    self.reject    )

# Save

def save(self):

    schedule = { "name": self.name_input.text(),

                "task_id":self.task_id,

                "start_date":self.date_input.date().toString("yyyy-MM-dd"),

                "start_time":self.time_input.time() .toString("HH:mm"),

                "repeat":self.repeat_combo.currentText(),

                "after_action":self.after_action_combo.currentText(),

                "enabled":self.enabled_check.isChecked()    }

    self.schedule_created.emit(schedule)

    self.accept()

```

3. scheduler.py

ဒီကောင်က အချိန်စောင့်မယ့် Engine။

PyQt6 မှာ အစမှာ QTimer သုံးရင် အလွယ်ဆုံး။

```

from PyQt6.QtCore import ( QObject, QTimer, QDateTime,)

class Scheduler(QObject):

    def __init__(    self,    schedule_manager,    download_manager    ):

```

```

super().__init__()

self.schedule_manager = (      schedule_manager      )

self.download_manager = (      download_manager      )

self.timer = QTimer()

self.timer.timeout.connect(      self.check_schedules      )

# Start

def start(self):

    # 1 second တစ်ခါ စစ်

    self.timer.start(1000)

# Stop

def stop(self):

    self.timer.stop()

# Check

def check_schedules(self):

    now = QDateTime.currentDateTime()

    current_date = now.date().toString(      "yyyy-MM-dd"      )

    current_time = now.time().toString(      "HH:mm"      )

    for schedule in (      self.schedule_manager.get_schedules()      ):

        if not schedule["enabled"]:

            continue

        if schedule["start_date"] != current_date:

            continue

        if schedule["start_time"] != current_time:

            continue

        self.run_schedule(      schedule      )

# Run

def run_schedule(self, schedule):

    task_id = schedule["task_id"]

```

```
print("Schedule Running:", schedule["name"] )
```

```
self.download_manager.start(task_id )
```

အခု Scheduler က—

တစ်စက္ကန့်တစ်ခါ

↓

Schedule စစ်

↓

Date မှန်လား?

↓

Time မှန်လား?

↓

Enabled လား?

↓

YES

↓

DownloadManager.start(task_id)

လုပ်မယ်။

4. MainWindow

MainWindow ကတော့ အားလုံးကို ချိတ်ပေးမယ်။

```
from PyQt6.QtWidgets import ( QMainWindow,)
```

```
from logic.schedule.schedule_manager import ( SchedulerManager)
```

```
from logic.schedule.scheduler import ( Scheduler)
```

```
from ui.dialogs.schedule_dialog import ( ScheduleDialog)
```

```
class MainWindow(QMainWindow):
```

```
    def __init__(self):
```

```

super().__init__()

self.setWindowTitle(      "Nexora"      )

self.resize(      900,      600      )

# Managers

self.schedule_manager = (      SchedulerManager()      )

# မင်းရဲ့ DownloadManager

self.download_manager = (      DownloadManager()      )

# Scheduler

self.scheduler = Scheduler(      self.schedule_manager,      self.download_manager
)

# Scheduler စ

self.scheduler.start()

# Open Schedule Dialog

def open_schedule_dialog(      self,      task_id=None      ):

    dialog = ScheduleDialog(      task_id,      self      )
    dialog.schedule_created.connect(      self.handle_schedule_created      )

    dialog.exec()

# Handle Schedule

def handle_schedule_created(      self,      data      ):

    schedule = (

        self.schedule_manager

        .create_schedule(

            name=data["name"],

            task_id=data["task_id"],

            start_date=data["start_date"],

            start_time=data["start_time"],

            repeat=data["repeat"],

            after_action=data["after_action"],

```

```

        enabled=data["enabled"]
    )

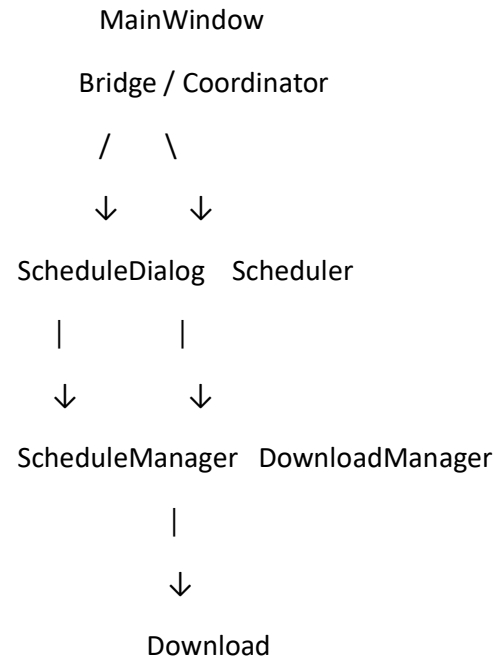
    print("Schedule Created:", schedule)

```

DownloadManager() က မင်း project ထဲမှာ ရှိပြီးသားဆို အဲ့ဒီ Manager ကို သုံးမယ်။

အခု Architecture

မင်းအခုလေ့လာနေတဲ့ **Bridge concept** နဲ့ တိတိကျကျကိုက်တယ်။



Schedule အသစ်ထည့်ရင်—

Schedule Button



MainWindow



ScheduleDialog



User Save



schedule_created



MainWindow



ScheduleManager



Schedule သိမ်း



Scheduler



အချိန်စောင့်



DownloadManager



Download Start

အခု version က learning/demo အတွက် အလုပ်လုပ်တဲ့ structure ဖြစ်တယ်။ Production

အတွက်တော့ နောက်တစ်ဆင့်မှာ ScheduleManager ကို SQLite နဲ့ချိတ်ပြီး၊

Daily/Weekly/Monthly repeat logic နဲ့ After Download action တွေကို

တကယ်အလုပ်လုပ်အောင် ထည့်ရမယ်။